



Заключительный этап Олимпиады школьников «Шаг в будущее»

Профиль: «Информатика»

Класс участия: 8-9

Задача 1 (8 баллов)

Комплекс приборов, отслеживающий состояние дрона, раз в секунду выдает натуральное число, записанное в десятичной системе счисления – код состояния узлов. Если цифра десятков в записи превышает цифру единиц, код считается критическим. После того, как комплекс перестаёт отправлять сигналы, вычисляется контрольное значение, которое рассчитывается так: к удвоенной сумме критических кодов добавляется количество некритических кодов, умноженное на 10^{11} . Располагая последовательностью кодов, определите контрольное значение.

Формат ввода

На вход программе в первой строке подаётся натуральное число N ($5 \leq N \leq 10000$) – длина последовательности кодов от комплекса. Далее в каждой строке подаётся целое число a_i ($0 \leq a_i \leq 1000$) – код состояния узлов в сигнале номер i .

Формат вывода

Вывести одно число – контрольное значение для этой серии испытаний.

Пример

Ввод	Вывод
5 789 934 781 923 576	300000002714

Решение:

```
n = int(input())
s = 0
kol = 0
for i in range(n):
    x = int(input())
    if ((x%100)//10)>(x%10):
        s+=2*x
    else:
        kol+=1
print(s+kol*10**11)
```



Критерии оценивания**

Критерий	Балл
Дан неверный ответ/ответ отсутствует	0
Дан верный ответ	8

** - Балл за каждую задачу проставляется в соответствии с долей выполнения участником задания. Шаг оценивания каждого задания - 1 балл. Доля выполнения вычисляется автоматической системой тестирования на основе комплекта установленных для задания тестов.

Задача 2 (12 баллов)

В начале суток на серверах запускаются несколько задач. Каждая задача выполняется целое количество минут. Так, если задача выполняется 15 минут, то она завершится в 00.15 ровно и ни мгновением раньше либо позже. Зная, какие задачи запущены, определите, сколько из них будет завершены в первой четверти какого-либо часа суток. Если время выполнения задачи выпадает на границу первой четверти, считать, что она выполнена за первую четверть часа.

Формат ввода

На вход программе в первой строке подаётся натуральное число N ($5 \leq N \leq 10000$) – количество задач, которые запускаются в начале суток. Далее в каждой строке подаётся натуральное число t_i ($1 \leq t_i \leq 1440$ мин) – время выполнения задачи номер i .

Формат вывода

Вывести одно целое число – сколько задач будут выполнены в первой четверти какого-либо часа суток.

Пример

Ввод	Вывод
5 607 15 22 60 19	3

Решение:

```
n = int(input())
s = 0
kol = 0
for i in range(n):
    x = int(input())
```



Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана (национальный исследовательский университет)»

ОЛИМПИАДА ШКОЛЬНИКОВ «ШАГ В БУДУЩЕЕ»

```
if ((x%100)//10)>(x%10):  
    s+=2*x  
else:  
    kol+=1  
print(s+kol*10**11)
```

Критерии оценивания**

Критерий	Балл
Дан неверный ответ/ответ отсутствует	0
Дан верный ответ	12

** - Балл за каждую задачу проставляется в соответствии с долей выполнения участником задания. Шаг оценивания каждого задания - 1 балл. Доля выполнения вычисляется автоматической системой тестирования на основе комплекта установленных для задания тестов.



Задача 3 (16 баллов)

Компания студентов решила организовать квест на празднике. У них есть коридор, разбитый на последовательность из комнат, пронумерованных от 1 до N (от входа к выходу). В каждой комнате стоит сундук с монетами. Игрок перемещается из комнаты в комнату строго в одном направлении.

На вход в квест (в комнату №1) игрок заходит при **зеленом** свете и с **нулевым** количеством монет.

В каждой комнате есть индикатор света (Зеленый, Желтый или Красный). Действия игрока и смена света для следующей комнаты зависят от текущего цвета света и содержимого сундука:

1. Зеленый свет:

- Если сундук **непустой**: игрок забирает все монеты себе. Свет в следующей комнате остается **Зеленым**.
- Если сундук **пустой**: игрок ничего не берет. Свет в следующей комнате становится **Желтым**.

2. Желтый свет:

- Если сундук **непустой**: игрок не может взять монеты. Свет в следующей комнате становится **Зеленым**.
- Если сундук **пустой**: свет в следующей комнате становится **Красным**.

3. Красный свет:

- Игрок обязан оставить в сундуке откуп (все накопленные монеты).
- Если у игрока **есть монеты**: он отдает их (баланс становится 0). Свет в следующей комнате становится **Зеленым**.
- Если у игрока **нет монет**: ему засчитывается **штраф**. Свет в следующей комнате остается **Красным**.

Условие победы: Игрок выигрывает, если он прошел все комнаты, не получил ни одного штрафа и вышел из коридора с количеством монет строго больше нуля. В противном случае игрок проигрывает.

Входные данные: В первой строке вводится целое число N ($0 < N < 1\,000\,000$) — количество комнат. Далее следует последовательность из N неотрицательных целых чисел — содержимое сундуков на пути от входа до выхода из коридора (каждое число на отдельной строке). Количество монет в одном сундуке не превышает 1000.

Выходные данные:

- Если игрок выиграл: в первой строке вывести латинскую букву V, во второй строке — итоговую сумму монет.
- Если игрок проиграл, то на первой строке выводится латинская буква N, а на второй номер комнаты, когда был получен **первый** штраф, или 0, если штрафа не было.



Примеры

Ввод	Вывод	Примечание
7 5 0 0 9 2 0 3	V 2	Цвет лампочек на пути игрока ЗЗЖКЗЗЖ Игрок заберет монетки в 1 комнате и оставит их в 4 комнате на откуп. В 5 комнате он возьмет 2 монетки, которые и составят его выигрыш. В последней комнате горит желтый свет, поэтому игрок не может взять монетки из сундука.
8 5 0 0 9 0 0 2 3	N 7	Цвет лампочек на пути игрока ЗЗЖКЗЖКК Игрок заберет монетки в 1 комнате и оставит их в 4 комнате на откуп. Больше монетки он собрать не сможет, так как на его пути будут пустые сундуки. В 7 комнате горит красный свет, а так как монет у игрока нет, то игрок получает штраф, в следующей - 8 комнате ситуация такая же.

Решение:

```
N = int(input())
f = False # штраф
ind = 0 # номер комнаты с первым штрафом
s = 0 # количество монет у игрока
col = 'green'
for i in range(N):
    x = int(input()) # содержимое сундука в текущей комнате
    if col == 'green':
        s += x
        if x == 0: col = 'yello'
    elif col == 'yello':
        if x == 0: col = 'red'
        else: col = 'green'
    elif col == 'red':
        if s > 0:
            col = 'green'
            s = 0
        else:
            if not f: ind = i + 1
            f = True
if f or s == 0:
    print('N')
    print(ind)
else:
    print('V')
    print(s)
```



Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана (национальный исследовательский университет)»

ОЛИМПИАДА ШКОЛЬНИКОВ «ШАГ В БУДУЩЕЕ»

Критерии оценивания**

Критерий	Балл
Дан неверный ответ/ответ отсутствует	0
Дан верный ответ	16

** - Балл за каждую задачу проставляется в соответствии с долей выполнения участником задания. Шаг оценивания каждого задания - 1 балл. Доля выполнения вычисляется автоматической системой тестирования на основе комплекта установленных для задания тестов.



Задача 4 (20 баллов)

На уроках информатики Фёдор узнал о методе шифрования RSA.

Он захотел придумать свой метод, который будет использовать пары простых чисел.

Фёдор решил, что пара простых чисел (p, q) будет "**подходящей**", если:

1. Количество цифр числа p (обозначим $K(p)$) и числа q ($K(q)$) удовлетворяют условию:

$$K(p) \leq K(q) \leq K(p) + 1.$$

2. Число t , полученное "склежкой" чисел p и q ($t = \text{Gluing}(p, q)$), также является простым числом.

Склежка" чисел a, b ($\text{Gluing}(a, b)$) — это получение нового числа, путем "дописывания" (конкатенации) к числу a числа b

Такие числа t Фёдор решил называть "**интересными**".

a	b	Gluing (a, b)
13	41	1341
2	11	211

Для каждого "интересного" числа t вычисляется значение $n(t) = p * q$.

Необходимо определить:

1. Количество "интересных" простых чисел на отрезке $[A, B]$.
2. Максимальное значение $n(t)$ среди всех "интересных" чисел на этом отрезке.

Входные данные:

В одной строке записаны через пробел два натуральных числа A и B ($10 \leq A \leq B < 10^7$).

Выходные данные:

Выведите два числа в отдельных строках:

1. Количество "интересных" чисел.
2. Максимальное значение $n(t)$.

Гарантируется, что на отрезке $[A, B]$ есть хотя бы одно "интересное" число.



ОЛИМПИАДА ШКОЛЬНИКОВ «ШАГ В БУДУЩЕЕ»

Примеры:

Входные данные	Ожидаемый результат	Пояснения
20 47	2 21	На отрезке [20,47] есть простые числа 23, 29, 31, 37, 41, 43, 47 Из них "интересными" будут только два: 2_3 и 3_7 Максимальное значение $n(t) = 3 \cdot 7 = 21$
100 300	6 166	"Интересными" будут числа 211, 223, 229, 241, 271, 283. Максимальное $n(t) = 2 \cdot 83 = 166$ даст число 283 Число 233 не будет интересным, так должно разбиваться только на пару 2_33, а не на пару 23_3
1000 9999	80 7387	

Решение:

```
def sieve_eratosfen (N):
```

```
    A = [False, False, True] + [True, False]*(N//2)
```

```
    kk = 2
```

```
    while kk * kk < N : kk += 1
```

```
    k = 3
```

```
    while k <= kk :
```

```
        if A[k]:
```

```
            for i in range (k*k, N+1, k):
```

```
                A[i] = False
```

```
            k += 2
```

```
    return [2] + [i for i in range(3, N+1, 2) if A[i]]
```

```
def check(n,P):
```

```
    s = str(n)
```

```
    k = len(s)//2
```

```
    a = int(s[:k])
```

```
    b = int(s[k:])
```

```
    if (s[k] != '0') and (a in P) and (b in P) : return a, b
```

```
    return 0 , 0
```

```
A, B = map(int,input().split())
```

```
Pr = sieve_eratosfen (B)
```

```
Spr = set(Pr)
```

```
l, r = 0,len(Pr)
```

```
while r - l > 1 :
```

```
    m = (r + l) // 2
```

```
    if Pr[m] < A : l = m
```

```
    else: r = m
```

```
cnt = 0
```

```
ans = 0
```

```
for n in Pr[r:]:
```

```
    a, b = check(n,Spr)
```

```
    if a == 0 : continue
```

```
    cnt += 1
```



ОЛИМПИАДА ШКОЛЬНИКОВ «ШАГ В БУДУЩЕЕ»

```
x = a * b
if x > ans :
    ans = x
n_ans = n
print(f'{cnt}\n{ans}')
```

Критерии оценивания**

Критерий	Балл
Дан неверный ответ/ответ отсутствует	0
Дан верный ответ	20

** - Балл за каждую задачу проставляется в соответствии с долей выполнения участником задания. Шаг оценивания каждого задания - 1 балл. Доля выполнения вычисляется автоматической системой тестирования на основе комплекта установленных для задания тестов.

Задача 5 (20 баллов)

Далекая цивилизация "Коллаторан" использовала для вычисления точки межзвёздного прыжка не линейные координаты, а уникальные числовые точки, именуемый "Точками Коллатца", генерируемые по таинственному алгоритму. Длительность полёта определялась количеством шагов, за которое число-ключ превращалось в единицу — точку сингулярности, откуда и начинался прыжок.

Ваш космический корабль оказался в секторе пространства, ограниченном некоторыми пространственными точками a и b , где a — колония цивилизации "Коллаторан", а b — главная планета. Чтобы выбрать оптимальную точку для самого долгого и впечатляющего прыжка, вам нужно проанализировать этот диапазон.

Условие:

Для двух ограничительных точек a и b ($1 \leq a \leq b \leq 1\,000\,000$) найдите точку Коллатца n на отрезке $[a, b]$, которая обладает самым длинным "звёздным путём" — последовательностью шагов по преобразованию Коллатца до достижения 1. Если таких точек будет несколько — выберите ту, которая будет ближе к главной планете цивилизации.

Алгоритм преобразования (шаг прыжка):

- Если число чётное: разделить его на 2.
- Если число нечётное: умножить его на 3 и прибавить 1.

Входные данные:

Два целых числа a и b , разделённые пробелом.

Выходные данные:

Два числа: искомое число n и длина его последовательности, разделённые пробелом.



Пример

Ввод	Вывод
10 20	19 20

*Пояснение к примеру – В диапазоне от 10 до 20 число 19 имеет самую длинную последовательность (20 шагов до достижения 1). Среди всех чисел этого диапазона его путь — самый долгий и зрелищный прыжок.

Решение:

```
MAX_LIMIT = 5000000
```

```
dp = [0] * (MAX_LIMIT + 1)
```

```
dp[1] = 0
```

```
def calc_length(n):
```

```
    if n < MAX_LIMIT and dp[n] != 0:
```

```
        return dp[n]
```

```
    chain = []
```

```
    current = n
```

```
    while current != 1 and (current >= MAX_LIMIT or dp[current] == 0):
```

```
        chain.append(current)
```

```
        if current % 2 == 0:
```

```
            current //= 2
```

```
        else:
```

```
            current = 3 * current + 1
```

```
    if current == 1:
```

```
        length = 0
```

```
    else:
```

```
        length = dp[current]
```

```
    for num in reversed(chain):
```

```
        length += 1
```

```
        if num < MAX_LIMIT:
```

```
            dp[num] = length
```

```
    return length
```

```
def solve():
```

```
    a, b = map(int, input().split())
```

```
    best_num = a
```

```
    best_len = 0
```

```
    for n in range(a, b + 1):
```

```
        l = calc_length(n)
```

```
        if l > best_len:
```

```
            best_len = l
```

```
            best_num = n
```

```
        elif l == best_len and n > best_num:
```

```
            best_num = n
```



Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана (национальный исследовательский университет)»

ОЛИМПИАДА ШКОЛЬНИКОВ «ШАГ В БУДУЩЕЕ»

```
print(f'{best_num} {best_len}')
```

```
solve()
```

Критерии оценивания**

Критерий	Балл
Дан неверный ответ/ответ отсутствует	0
Дан верный ответ	20

** - Балл за каждую задачу проставляется в соответствии с долей выполнения участником задания. Шаг оценивания каждого задания - 1 балл. Доля выполнения вычисляется автоматической системой тестирования на основе комплекта установленных для задания тестов.



Задача 6 (24 балла)

Геологическая экспедиция исследует отдалённый район на Крайнем Севере. Для нормальной работы им необходимо топливо. Работа уже распланирована, и заранее известно, сколько топлива им понадобится на каждый день работы. Часть топлива они берут с собой, выезжая в район. Остальное топливо им будут подвозить во время работы. Существует несколько вариантов поставки топлива, которые отличаются объёмом и периодом подвоза. Определите количество вариантов поставки топлива, при которых экспедиция может работать без простоев. Следует учитывать, что:

- 1) Простой возникает, когда не хватает топлива на день.
- 2) Весь объём топлива на день берут в начале суток; если на складе 200 литров, а требуется на день 300, будет простой.
- 3) Топливо подвозят в конце суток, и оно будет готово к использованию на следующие сутки; если на складе было 200 литров, на день требовалось 150, а подвезли 300, то на следующий день доступны 350.
- 4) Вариант поставки объёмом M с периодом завоза T дней означает, что первая поставка объёмом M , произойдёт в день номер T , потом привезут такой же объём в день номер $2T$ и т. д.; нумерация дней идёт с единицы.

Формат ввода

На вход программе в первой строке подаётся натуральное число N ($N \leq 10000$) – количество дней работы экспедиции. Во второй строке подаётся натуральное число K ($K \leq 10000$) – количество вариантов поставки топлива. В третьей строке подаётся натуральное число M_0 ($M_0 \leq 1000000$ л) – запас топлива в начале работы экспедиции. Далее в K строках подаётся по два разделённых пробелом натуральных числа M_j и t_j ($M_j \leq 1000000$ л, $t_j \leq 100$ д) – объём и период поставки в варианте номер j . Далее в N строках подаётся неотрицательное число m_i ($m_i \leq 100000$ л) – объём топлива, который нужен экспедиции в день номер i .

Формат вывода

Вывести одно целое число – количество вариантов поставки топлива, при которых экспедиция может работать без простоев.

Пример

Ввод	Вывод
5	1
2	
30	
3 1	
15 2	
15	
16	
1	
1	
2	



Решение:

```
def good(zapas,tanker):  
    n = tanker[1]  
    plus = tanker[0]  
    balance = [zapas[0]]  
    for i in range(1,len(zapas)):  
        balance.append(balance[-1]+zapas[i])  
        if balance[-1] < 0:  
            return 0  
        if i%n==0:  
            balance[-1]+=plus  
    return 1
```

```
n = int(input())  
k = int(input())  
zapas = [int(input())]  
oil = []  
for i in range(k):  
    tanker = list(map(int,input().split()))  
    oil.append(tanker)  
    #print(fracht)  
for _ in range(n):  
    zapas.append(-int(input()))  
answer = 0  
for i in range(k):  
    answer+=good(zapas,oil[i])  
  
print(answer)
```

Критерии оценивания**

Критерий	Балл
Дан неверный ответ/ответ отсутствует	0
Дан верный ответ	24

** - Балл за каждую задачу проставляется в соответствии с долей выполнения участником задания. Шаг оценивания каждого задания - 1 балл. Доля выполнения вычисляется автоматической системой тестирования на основе комплекта установленных для задания тестов.